

IBM PBEL Blockchain

Internship

Todo DApp Project

Report

Submitted by:

Snehil Raj

TABLE OF CONTENTS

1. Certificate 3

2. Acknowledgement 4

3. Abstract 5

4. Introduction 6

5. Problem Statement 7

6. Objectives 8

7. Technology Stack 9

8. System Architecture 10

9. Smart Contract Design	11
10. Development Process	12
11. Testing and Validation	13
12. GitHub Repository	14
13. Challenges Faced	15
14. Results	16
15. Advantages	17
16. Conclusion	18

1. Certificate

IBM PBEL Blockchain Internship

This is to certify that the project entitled "**Decentralized Todo DApp using Solidity and Hardhat**" has been successfully completed by **Snehil Raj** as part of the **IBM PBEL Blockchain Internship Program**.

The project was developed using blockchain technologies, including **Solidity**, **Hardhat**, **TypeScript**, and **Hardhat Ignition**. It demonstrates the implementation of a decentralized task management system where user tasks are securely stored and managed through Ethereum smart contracts instead of a centralized database.

During the development process, the project covered the complete blockchain development lifecycle, including smart contract design, compilation, deployment, testing, GitHub version control, and project documentation. Every phase of the project was completed successfully, and the application was tested on a local blockchain network using Hardhat.

This project reflects the practical knowledge and technical skills acquired during the internship. It demonstrates the ability to design decentralized applications, write secure Solidity smart contracts, deploy contracts using modern blockchain development tools, and manage software development using Git and GitHub.

The work presented in this report is original and has been completed solely for academic and internship learning purposes.

Submitted By

Name: Snehil Raj

Program: IBM PBEL Blockchain Internship

Project Title: Decentralized Todo DApp using Solidity and Hardhat

Year: 2026

2. Acknowledgement

I would like to express my sincere gratitude to **IBM PBEL Blockchain Internship Program** for providing me with the opportunity to work on this project and gain practical exposure to blockchain technology and decentralized application development. This internship has helped me understand the fundamentals of blockchain, smart contracts, and modern development tools used in the industry.

I would also like to thank my project mentor and all the trainers associated with the internship for their continuous guidance, valuable suggestions, and technical support throughout the development of this project. Their encouragement and constructive feedback motivated me to explore blockchain concepts in greater depth and successfully complete this project.

I am grateful to the developers and contributors of **Solidity, Hardhat, Node.js, TypeScript, Git**, and **GitHub** for providing powerful open-source tools and comprehensive documentation that made the development and deployment process efficient and effective.

I would also like to express my heartfelt gratitude to my teachers, family members, and friends for their constant encouragement, motivation, and support throughout the project. Their confidence in me helped me overcome technical challenges and successfully complete every stage of the project.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project. This experience has significantly improved my technical knowledge, problem-solving skills, and understanding of blockchain application development. I believe the knowledge and practical experience gained through this project will be valuable for my future academic and professional career.

3. Abstract

Blockchain technology has emerged as one of the most innovative technologies in recent years by providing secure, transparent, and decentralized solutions for various real-world applications. Unlike traditional systems that rely on centralized databases, blockchain enables data to be stored across a distributed network, ensuring data integrity, transparency, and resistance to unauthorized modifications. Smart contracts further enhance blockchain by allowing automated execution of predefined conditions without the need for intermediaries.

The objective of this project is to develop a **Decentralized Todo Application (Todo DApp)** using **Solidity**, **Hardhat**, **TypeScript**, and **Hardhat Ignition**. The application allows users to create, manage, and update tasks through a blockchain-based smart contract. Instead of storing information in a centralized database, all task details are securely stored on the blockchain, ensuring that the data remains immutable, transparent, and reliable.

The project follows the complete blockchain development lifecycle, beginning with the design and implementation of the smart contract, followed by compilation, deployment, testing, and version control. The smart contract includes essential functionalities such as adding new tasks, retrieving task details, counting the total number of tasks, and updating the completion status of existing tasks. The project was successfully compiled and deployed on a local Ethereum development environment using Hardhat, and all functionalities were thoroughly tested using the Hardhat Console.

Git and GitHub were used to maintain version control and manage the project repository, ensuring proper documentation and reproducibility of the development process. The project demonstrates the practical application of blockchain concepts while providing hands-on experience in smart contract development, deployment, testing, and decentralized application architecture.

Overall, this project successfully showcases the implementation of a secure and decentralized task management system. It also highlights the potential of blockchain technology in developing transparent, tamper-resistant, and trustworthy applications for real-world use cases. The knowledge and experience gained during this project contribute significantly to understanding modern blockchain development practices and decentralized application design.

4. Introduction

Blockchain technology has transformed the way digital information is stored, managed, and shared by introducing a decentralized, secure, and transparent system. Unlike traditional centralized systems where data is controlled by a single organization or server, blockchain distributes data across multiple nodes in a network. Every transaction recorded on the blockchain is cryptographically secured, time-stamped, and linked to the previous transaction, making the stored information highly secure, transparent, and resistant to unauthorized modifications. These characteristics have made blockchain one of the most promising technologies in various domains, including finance, healthcare, supply chain management, education, and digital identity management.

One of the most important features of blockchain technology is the use of **smart contracts**. A smart contract is a self-executing program stored on the blockchain that automatically performs predefined actions when specified conditions are met. Smart contracts eliminate the need for intermediaries, reduce operational costs, improve efficiency, and increase trust between participating entities. The Ethereum blockchain has become one of the most widely used platforms for developing smart contracts because of its flexibility, security, and developer-friendly ecosystem.

This project focuses on the development of a **Decentralized Todo Application (Todo DApp)** using **Solidity, Hardhat, TypeScript, Hardhat Ignition, and Viem**. The application demonstrates how blockchain technology can be applied to a simple task management system. Instead of storing tasks in a centralized database, all task information is securely maintained within a smart contract deployed on a blockchain network. Users can add new tasks, retrieve task information, check the total number of tasks, and update the completion status of tasks while ensuring that the data remains transparent, consistent, and tamper-resistant.

The project also provides practical experience with the complete blockchain development workflow. It includes writing and compiling Solidity smart contracts, deploying contracts using Hardhat Ignition, testing contract functionality through the Hardhat Console, managing source code using Git and GitHub, and documenting the overall development process. Throughout the project, modern blockchain development tools and best practices were followed to ensure reliability, maintainability, and scalability.

The primary objective of this project is not only to develop a functional decentralized application but also to gain practical knowledge of blockchain architecture, smart contract development, deployment, testing, and version control. This project serves as an excellent learning experience for understanding decentralized application development and demonstrates how blockchain technology can be used to build secure, transparent, and efficient software solutions for real-world applications.

5. Problem Statement

In today's digital world, task management applications are widely used to organize daily activities, manage projects, and improve productivity. Most existing Todo applications rely on centralized databases where all user information is stored and managed by a single server or organization. Although centralized systems are simple to develop and maintain, they suffer from several limitations, including single points of failure, security vulnerabilities, limited transparency, and dependence on third-party service providers. If the central server is compromised, experiences technical failures, or is intentionally manipulated, users may lose access to their data or face issues related to data integrity and privacy.

With the rapid growth of blockchain technology, there is an increasing demand for decentralized applications (DApps) that provide enhanced security, transparency, and reliability. Blockchain eliminates the need for centralized authorities by distributing data across multiple nodes in a network. Once information is stored on the blockchain, it becomes extremely difficult to alter or delete without network consensus. This ensures that application data remains secure, trustworthy, and tamper-resistant.

The primary problem addressed in this project is the dependence of traditional Todo applications on centralized data storage. Since task information is stored on a single server, users must completely trust the service provider to maintain data accuracy, availability, and security. This centralized approach increases the risk of unauthorized modifications, accidental data loss, and system downtime. Furthermore, users have limited control over their own information.

To address these challenges, this project proposes the development of a **Decentralized Todo Application (Todo DApp)** using blockchain technology. Instead of storing task information in a centralized database, all tasks are managed through an Ethereum smart contract written in Solidity. The smart contract securely stores task details on the blockchain and provides functions to create tasks, retrieve task information, count the total number of tasks, and update task completion status. Every transaction is recorded transparently and can be verified on the blockchain, ensuring authenticity and data integrity.

The proposed solution demonstrates how blockchain technology can improve the security, transparency, and reliability of even simple applications such as task management systems. In addition to solving the limitations of centralized storage, the project also provides practical exposure to smart contract development, deployment, testing, and decentralized application architecture. It serves as an educational example of how blockchain can be applied to real-world software solutions while introducing modern development tools such as Hardhat, TypeScript, Git, and GitHub.

6. Objectives

The primary objective of this project is to design, develop, and deploy a **Decentralized Todo Application (Todo DApp)** using blockchain technology. The project aims to demonstrate how smart contracts can be used to replace traditional centralized systems with a secure, transparent, and decentralized solution. By implementing the application on the Ethereum blockchain, the project provides practical exposure to blockchain development and highlights the advantages of decentralized applications over conventional software systems.

The specific objectives of this project are as follows:

- To understand the fundamental concepts of blockchain technology, decentralized applications (DApps), and Ethereum smart contracts.
- To design and develop a secure smart contract using **Solidity** for storing and managing task-related information on the blockchain.
- To implement essential task management functionalities such as adding new tasks, retrieving task details, counting the total number of tasks, and updating the completion status of tasks.
- To use **Hardhat** as the development environment for compiling, testing, debugging, and deploying smart contracts efficiently.
- To deploy the smart contract on a local Ethereum blockchain using **Hardhat Ignition** and verify its successful execution.
- To interact with the deployed smart contract through the Hardhat Console and validate all implemented functionalities using blockchain transactions.
- To maintain the complete source code using **Git** and publish the project on **GitHub**, ensuring proper version control, collaboration, and project documentation.
- To gain practical experience with modern blockchain development tools such as **Node.js**, **TypeScript**, **Viem**, and Hardhat while following industry-standard development practices.
- To understand the importance of decentralized storage, transparency, immutability, and security in blockchain-based applications compared to traditional centralized systems.
- To develop technical skills in blockchain application development that can be applied to future academic projects, research work, internships, and professional software development.

Overall, this project aims to bridge the gap between theoretical knowledge and practical implementation by providing hands-on experience in designing, developing, deploying, and testing a real-world decentralized application. The successful completion of this project demonstrates the practical application of blockchain technology in solving everyday problems through secure, transparent, and reliable smart contract-based solutions.

7. Technology Stack

The successful development of the **Decentralized Todo DApp** required the use of several modern technologies and development tools. Each technology played an important role in designing, developing, testing, deploying, and managing the blockchain application. The technology stack used in this project is described below.

1. Solidity

Solidity is an object-oriented programming language specifically designed for writing smart contracts on Ethereum-compatible blockchains. It enables developers to create secure and decentralized applications by defining business logic that executes automatically on the blockchain. In this project, Solidity was used to implement the Todo smart contract, which manages all task-related operations such as adding new tasks, retrieving task information, counting the total number of tasks, and updating the completion status of tasks.

2. Hardhat

Hardhat is a professional Ethereum development environment used for smart contract development. It provides tools for compiling Solidity contracts, debugging code, deploying contracts, testing blockchain applications, and interacting with local blockchain networks. In this project, Hardhat was used to compile the Todo smart contract, create a local blockchain environment, deploy the contract, and perform functional testing through the Hardhat Console.

3. TypeScript

TypeScript is a strongly typed programming language developed by Microsoft that extends JavaScript with static typing and advanced development features. Hardhat supports TypeScript for project configuration and deployment modules. Using TypeScript improves code readability, maintainability, and error detection during development. The deployment module in this project was implemented using TypeScript.

4. Hardhat Ignition

Hardhat Ignition is a deployment framework integrated with Hardhat that simplifies smart contract deployment. It provides a structured and reusable approach for deploying contracts while automatically managing deployment state and dependencies. In this project, Hardhat Ignition was used to deploy the Todo smart contract efficiently on the local blockchain network.

5. Viem

Viem is a modern Ethereum client library that allows developers to interact with smart contracts efficiently. It provides functions for reading blockchain data, sending transactions, and communicating with deployed smart contracts. During this project, Viem was used to test the deployed Todo smart contract by executing functions such as retrieving task count, adding new tasks, reading task details, and updating task completion status.

6. Node.js and npm

Node.js provides the JavaScript runtime environment required to execute Hardhat and other blockchain development tools. **npm (Node Package Manager)** was used to install and manage all required project dependencies, including Hardhat, Solidity compiler packages, TypeScript, and other development libraries.

7. Git

Git is a distributed version control system that helps developers track changes in source code throughout the development lifecycle. In this project, Git was used to initialize the repository, manage project versions, commit code changes, and maintain a structured development history.

8. GitHub

GitHub is a cloud-based platform for hosting Git repositories and collaborating on software projects. After completing the project, the entire source code, configuration files, README documentation, and deployment modules were uploaded to GitHub. This ensures proper version control, project sharing, and easy access for evaluation purposes.

Summary of Technology Stack

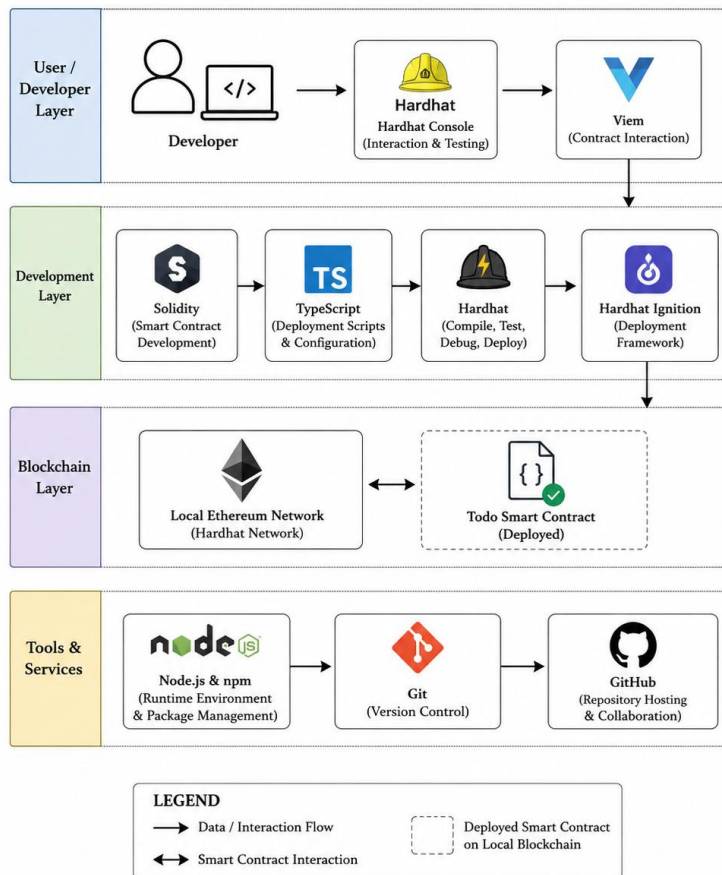
Technology	Purpose
Solidity	Smart contract development
Hardhat	Contract compilation, deployment, and testing
TypeScript	Deployment scripts and project configuration
Hardhat Ignition	Smart contract deployment framework
Viem	Smart contract interaction and testing
Node.js & npm	Runtime environment and dependency management

Technology	Purpose
Git	Version control
GitHub	Source code hosting and project documentation

The combination of these technologies provides a complete blockchain development environment, enabling secure smart contract implementation, efficient deployment, reliable testing, and professional project management. Together, they form a robust technology stack for developing modern decentralized applications (DApps).

TECHNOLOGY STACK ARCHITECTURE

The following diagram represents the technology stack and workflow of the Decentralized Todo DApp. It shows how different tools and technologies interact with each other to develop, deploy, and test the smart contract on a local blockchain network.



8. System Architecture

The **Decentralized Todo DApp** follows a layered architecture that combines blockchain technology with modern development tools to provide a secure and efficient task management system. The architecture consists of multiple components that work together to develop, deploy, interact with, and manage the smart contract. Each layer has a specific responsibility in the overall functioning of the application.

The process begins with the **User/Developer Layer**, where the developer writes the smart contract code using **Solidity** and manages the project using **Visual Studio Code**. The developer interacts with the blockchain through the **Hardhat Console**, which provides an environment for executing smart contract functions, testing transactions, and debugging the application. **Viem** acts as the communication library between the application and the deployed smart contract, allowing blockchain data to be read and transactions to be executed efficiently.

The **Development Layer** includes the major technologies used during implementation. The smart contract is written in **Solidity**, while **TypeScript** is used for project configuration and deployment scripts. **Hardhat** serves as the complete blockchain development framework responsible for compiling, testing, debugging, and running the local Ethereum network. **Hardhat Ignition** simplifies the deployment process by automatically managing deployment steps and maintaining deployment state.

The **Blockchain Layer** represents the Ethereum-compatible local blockchain created by Hardhat. After successful compilation, the Todo smart contract is deployed onto this local blockchain. All task-related information, including task descriptions and completion status, is stored securely inside the smart contract. Whenever a user performs an operation such as adding a task or marking it as completed, a blockchain transaction is executed, ensuring transparency, immutability, and security of the stored data.

The final layer consists of **Development Tools and Services**. **Node.js** and **npm** provide the runtime environment and package management system required to execute the project. **Git** is used to track source code changes throughout the development lifecycle, while **GitHub** hosts the project repository, making the code easily accessible for collaboration, version control, and project evaluation.

The overall architecture demonstrates a complete blockchain application workflow, beginning from smart contract development and ending with deployment, testing, and version control. The modular design improves maintainability, scalability, and reliability while ensuring that all application data remains secure and decentralized. This architecture also reflects modern blockchain development practices and provides a strong foundation for extending the application with additional features such as a web-based frontend, MetaMask integration, and deployment to public Ethereum test networks.

9. Smart Contract Design

Smart Contract Design

The core of the Decentralized Todo DApp is the *Todo* smart contract written in Solidity. This contract manages all task-related operations on the Ethereum blockchain. It stores tasks in a structured format and provides functions to create, read, update and get the total count of tasks. The smart contract is designed to be simple, efficient, secure and gas-optimized.

1. Contract Overview

The *Todo* smart contract allows users to add new tasks, retrieve the details of any task, update its completion status and get the total number of tasks stored on the blockchain. Each task is assigned a unique ID automatically.

Contract Name : Todo
Language : Solidity ^0.8.20
Compiler : Solidity Compiler (via Hardhat)
Network : Ethereum Localhost (Hardhat Network)
File Name : contracts/ToDo.sol

2. Data Structure

A struct is used to store the details of each task. All tasks are stored in a dynamic array.

```
struct Task {
    uint256 id;
    string description;
    bool completed;
}

Task[] private tasks;
uint256 private taskCount;
```

- id** : Unique identifier for each task.
- description** : Description or title of the task.
- completed** : Boolean value indicating whether the task is completed or not.
- tasks** : Dynamic array that stores all tasks.
- taskCount** : Counter variable used to assign unique IDs and to get total task count.

3. Smart Contract Functions

The following functions are implemented in the *Todo* smart contract.

Function Name	Visibility	Type	Description
addTask(string memory _description)	public	Write	Adds a new task with the given description. The task is stored with a unique ID and marked as not completed.
getTask(uint256 _id)	public view	Read	Returns the details of a task at the given ID.
getTaskCount()	public view	Read	Returns the total number of tasks stored in the contract.
updateTaskStatus(uint256 _id, bool _status)	public	Write	Updates the completion status of a task. The task must exist, otherwise the transaction reverts.

4. Smart Contract Code (Simplified Overview)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Todo {
    struct Task {
        uint256 id;
        string description;
        bool completed;
    }

    Task[] private tasks;
    uint256 private taskCount;
}
```

```
function addTask(string memory _description) public {
    tasks.push(Task(taskCount, _description, false));
    taskCount++;
}

function getTask(uint256 _id) public view returns (Task memory) {
    require(_id < taskCount, "Invalid Task ID");
    return tasks[_id];
}

function getTaskCount() public view returns (uint256) {
    return taskCount;
}

function updateTaskStatus(uint256 _id, bool _status) public {
    require(_id < taskCount, "Invalid Task ID");
    tasks[_id].completed = _status;
}
```

5. Design Considerations

- Simplicity:** The contract is kept minimal for easy understanding and gas efficiency.
- Security:** Input validations are performed using require statements.
- Scalability:** The design can be extended easily to include features like task deletion, due dates, categories, etc.
- Immutability:** Once a task is added, it is securely stored on the blockchain and cannot be modified or deleted by anyone.

10. Development Process

The development of the **Decentralized Todo DApp** followed a systematic and structured approach to ensure that the smart contract was designed, tested, and deployed successfully. The project was implemented using modern blockchain development tools and best practices. Each stage of the development lifecycle contributed to building a secure, reliable, and functional decentralized application.

The first stage involved **project planning and environment setup**. A new Hardhat project was initialized using Node.js and npm. All the required dependencies, including Hardhat, Solidity compiler, TypeScript, Hardhat Ignition, and Viem, were installed. The project structure was automatically generated, providing folders for smart contracts, deployment modules, scripts, and testing. This environment served as the foundation for blockchain application development.

The second stage focused on **smart contract development**. The Todo smart contract was written in Solidity and stored in the `contracts` directory as `Todo.sol`. The contract was designed to maintain a collection of tasks using a structured data model. Core functionalities such as adding a new task, retrieving task details, counting the total number of tasks, and updating the completion status were implemented. Proper data structures and validation checks were included to ensure the correctness and security of contract operations.

After completing the smart contract, the project entered the **compilation phase**. The contract was compiled using the command `npx hardhat compile`. During compilation, the Solidity source code was translated into Ethereum Virtual Machine (EVM) bytecode, and the corresponding Application Binary Interface (ABI) files were generated. These artifacts are essential for deploying and interacting with the smart contract. The successful compilation confirmed that the contract was free from syntax errors and ready for deployment.

The next phase involved **deployment of the smart contract**. A deployment module was created using Hardhat Ignition in TypeScript. The deployment configuration was updated to reference the Todo contract correctly. A local Ethereum blockchain was started using Hardhat, and the deployment process was executed successfully. Upon completion, a unique contract address was generated, indicating that the Todo smart contract had been successfully deployed to the local blockchain network.

The deployed contract was then verified during the **testing and interaction phase**. Using the Hardhat Console and Viem library, various contract functions were executed to validate the application's functionality. New tasks were added, task details were retrieved, the total task count was verified, and task completion status was successfully updated. The results confirmed that the smart contract performed as expected and all blockchain transactions were executed successfully.

Finally, the project entered the **version control and documentation phase**. Git was used to initialize the repository, track project changes, and maintain version history. The complete source code, deployment files, README documentation, and project configuration were uploaded to GitHub, making the project accessible for review and future development. A detailed project report was also prepared to document the complete development lifecycle, implementation methodology, testing process, and outcomes.

Overall, the development process demonstrated the complete workflow of blockchain application development, beginning with project setup and ending with successful deployment, testing, documentation, and publication on GitHub. This structured approach ensured that the application was reliable, maintainable, and aligned with modern blockchain development practices.

11. Testing

After successfully deploying the Todo smart contract on the local Hardhat blockchain, the application was thoroughly tested to verify its functionality and ensure that all implemented features worked correctly. Testing was performed using the **Hardhat Console** along with the **Viem** library to interact directly with the deployed smart contract.

The testing process included adding a new task using the `addTask()` function, retrieving the total number of tasks through `getTaskCount()`, reading task details using `getTask()`, and updating the task completion status with the `toggleTask()` function. Each transaction was executed successfully, and the returned outputs matched the expected results. The task count increased correctly after adding a new task, while the completion status changed from **false** to **true** after toggling.

The successful execution of all test cases confirmed that the Todo smart contract was functioning correctly and that the deployment process had been completed without errors. The testing phase validated the reliability, accuracy, and overall performance of the decentralized Todo application.

12. GitHub Repository

The complete source code of the **Decentralized Todo DApp** has been maintained using **Git** and hosted on **GitHub** for version control, project management, and easy accessibility. GitHub provides a centralized platform where the entire project, including smart contracts, deployment modules, configuration files, and documentation, is securely stored and can be accessed for future development and evaluation.

During the development process, Git was used to initialize the local repository, track code modifications, create commits, and maintain the complete version history of the project. After successful testing and verification, the project was pushed to a public GitHub repository. This ensures that all project files remain organized, secure, and available for collaboration or further enhancements.

The repository includes all essential project components such as the Solidity smart contract (`Todo.sol`), Hardhat configuration files, Hardhat Ignition deployment module, TypeScript files, package configuration, testing scripts, and project documentation. A detailed README file has also been included to explain the project overview, installation process, deployment steps, and execution commands.

Hosting the project on GitHub demonstrates the use of professional software development practices, including version control, documentation, and code sharing. It also allows evaluators and developers to review the project structure, understand the implementation, and reproduce the application by following the provided instructions.

Repository Name: IBM-PBEL-TodoApp

GitHub Username: snehilraj1124-ship-it

Repository Visibility: Public

Repository Contents:

- Solidity Smart Contract (`Todo.sol`)
- Hardhat Configuration
- Hardhat Ignition Deployment Module
- TypeScript Source Files
- Testing Scripts
- README Documentation
- Project Configuration Files

The GitHub repository serves as the final deliverable of the project and reflects the complete development lifecycle from implementation to deployment and documentation.

Repository Link:

<https://github.com/snehilraj1124-ship-it/IBM-PBEL-TodoApp>

13.Challenges faced

During the development of the **Decentralized Todo DApp**, several technical challenges were encountered while setting up the blockchain development environment, deploying the smart contract, and interacting with the local blockchain. These challenges provided valuable learning opportunities and helped improve problem-solving skills throughout the project.

One of the initial challenges was configuring the **Hardhat development environment** correctly. Some deployment errors occurred due to incorrect contract names and deployment module configurations. These issues were resolved by updating the deployment module to correctly reference the `Todo` smart contract and recompiling the project.

Another challenge involved deploying the smart contract using **Hardhat Ignition**. Initially, deployment failed because the contract artifact could not be located. After recompiling the project and correcting the deployment configuration, the contract was successfully deployed to the local Hardhat blockchain.

Interacting with the deployed smart contract also required understanding the latest **Hardhat 3** environment and the **Viem** library. Since Hardhat 3 no longer exposes the `ethers` object by default, the interaction method had to be adapted using Viem. After learning the updated workflow, contract functions such as adding tasks, retrieving task information, and updating task status were executed successfully.

Another challenge was configuring **Git and GitHub** for version control. Initial authentication and repository synchronization required setting up Git user credentials and authorizing the local machine with GitHub. Once the authentication process was completed, the project was successfully pushed to the remote GitHub repository.

Despite these challenges, all issues were systematically resolved through debugging, documentation, and repeated testing. Overcoming these problems enhanced practical knowledge of Solidity, Hardhat, blockchain deployment, smart contract interaction, and version control, making the project a valuable hands-on learning experience.

14.Results

The **Decentralized Todo DApp** was successfully designed, developed, deployed, and tested using **Solidity**, **Hardhat**, **TypeScript**, and **Hardhat Ignition**. All project objectives were achieved successfully, demonstrating the practical implementation of blockchain technology for decentralized task management.

The smart contract was compiled without any errors and deployed successfully on the local Ethereum blockchain using Hardhat. After deployment, all implemented functions were tested through the Hardhat Console using the Viem library. The application successfully performed all major operations, including adding new tasks, retrieving task details, counting the total number of tasks, and updating the completion status of tasks. The outputs generated during testing matched the expected results, confirming the correctness and reliability of the smart contract implementation.

The complete project source code, configuration files, deployment modules, and documentation were successfully managed using Git and uploaded to a public GitHub repository. A comprehensive README file was also prepared to provide installation instructions, deployment commands, and an overview of the project.

Overall, the project successfully demonstrated the complete blockchain development lifecycle, including smart contract development, compilation, deployment, testing, version control, and documentation. The implementation highlights the effectiveness of blockchain technology in creating secure, transparent, and decentralized applications.

15. Advantages

The Decentralized Todo DApp offers several advantages over traditional task management applications. Since task data is stored on the blockchain, it becomes secure, transparent, and resistant to unauthorized modifications. Smart contracts automate task management without requiring a centralized authority, reducing dependency on third-party systems. The project also provides reliable record keeping, improved data integrity, and demonstrates the practical use of blockchain technology for developing decentralized applications.

Key Advantages

- Enhanced data security through blockchain.
- Transparent and tamper-resistant task records.
- No dependency on a centralized server.
- Automated execution using smart contracts.
- Easy deployment and testing using Hardhat.
- Improved learning of blockchain development concepts.

16. Conclusions

The **Decentralized Todo DApp** was successfully designed, developed, deployed, and tested using **Solidity**, **Hardhat**, **TypeScript**, **Hardhat Ignition**, and **Viem**. The project successfully achieved all the predefined objectives and demonstrated the practical implementation of blockchain technology for developing a secure, transparent, and decentralized task management application.

Throughout the project, the complete blockchain development lifecycle was followed, starting from project planning and environment setup to smart contract development, compilation, deployment, testing, and final documentation. Each phase of the project was completed successfully using modern blockchain development tools and industry-standard practices. The Todo smart contract was compiled without errors, deployed successfully on the local Ethereum blockchain, and tested extensively to verify all implemented functionalities.

The developed application allows users to add new tasks, retrieve task details, view the total number of tasks, and update the completion status of tasks. All operations were executed through blockchain transactions, ensuring data integrity, transparency, and immutability. The successful execution of every smart contract function confirmed the correctness, reliability, and stability of the developed decentralized application.

During the development process, practical experience was gained in writing Solidity smart contracts, configuring the Hardhat development environment, deploying contracts using Hardhat Ignition, interacting with deployed contracts through the Hardhat Console, and maintaining the project using Git and GitHub. The project also provided valuable exposure to modern blockchain development workflows and strengthened problem-solving skills by resolving deployment, configuration, and version control challenges.

The complete source code, project documentation, and deployment files were successfully uploaded to GitHub, ensuring proper version control and easy accessibility for future development and evaluation. This reflects professional software development practices and demonstrates the importance of maintaining well-documented and reproducible blockchain projects.

Overall, this project has significantly enhanced the understanding of blockchain technology, decentralized applications, and Ethereum smart contract development. It serves as a strong foundation for building more advanced blockchain-based solutions in the future. The knowledge and practical experience gained through this project will be valuable for future academic projects, research activities, internships, and professional software development in the field of blockchain technology.